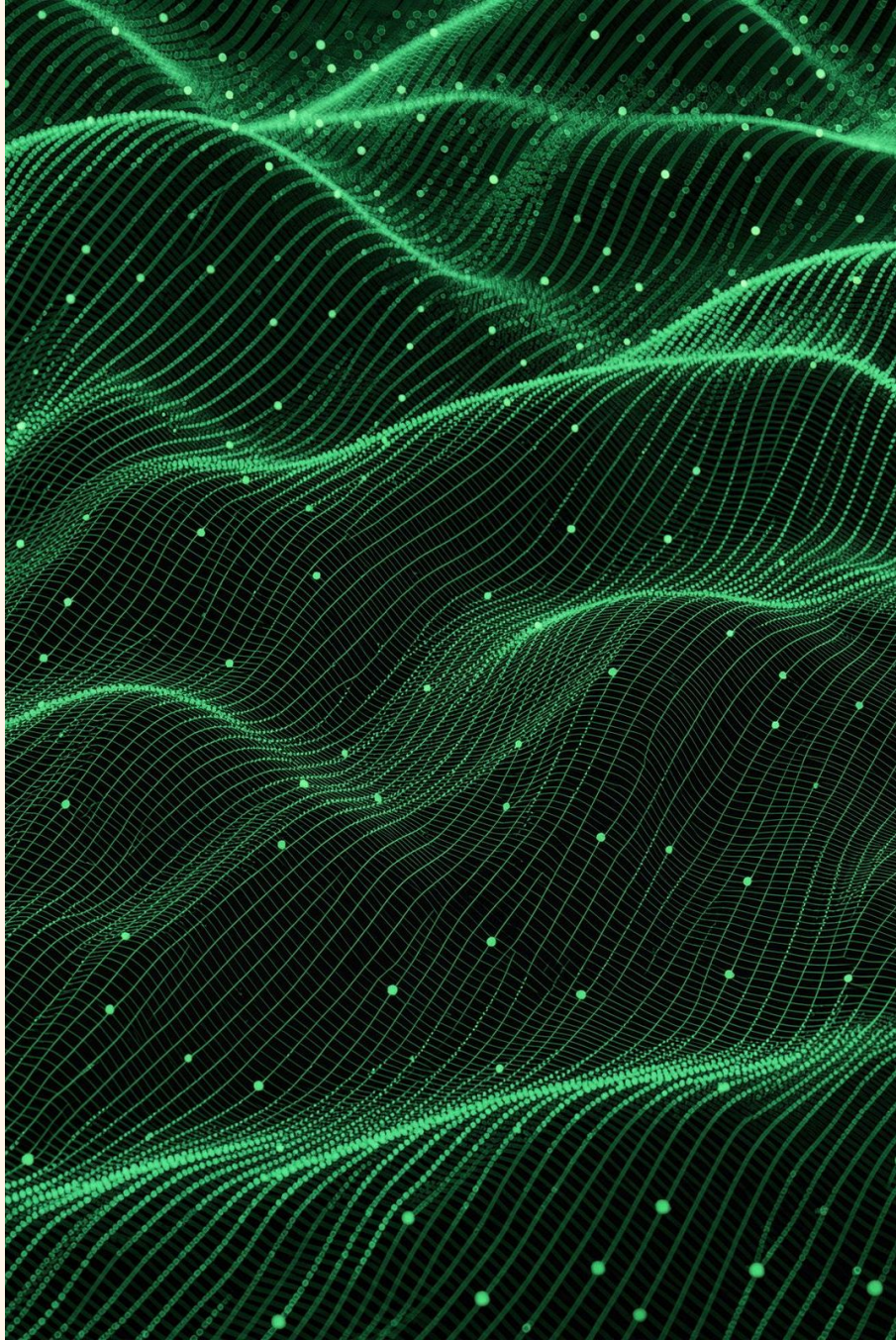
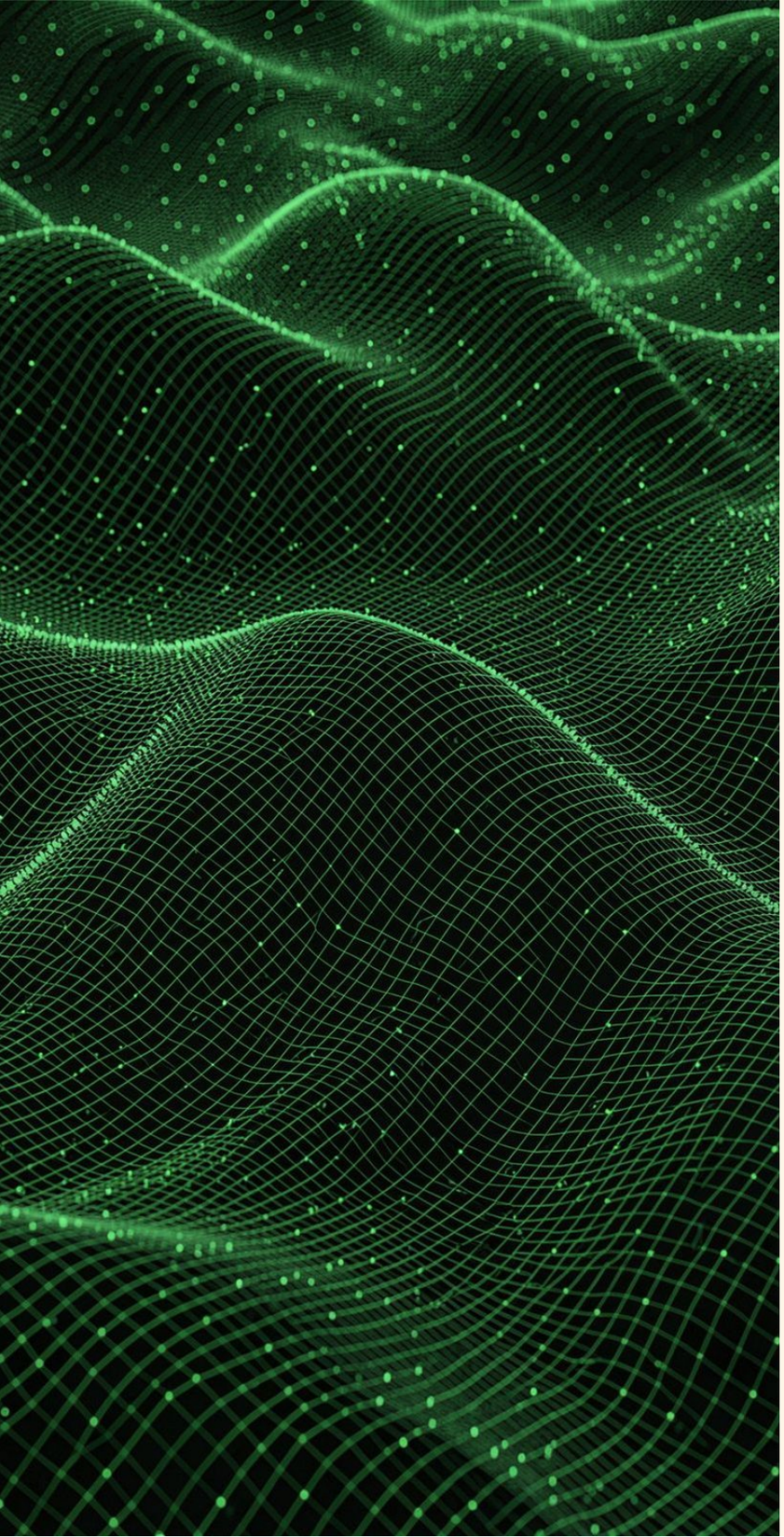
CSS Fundamentals: Margin, Padding, Border & Beyond

A classroom-ready guide to mastering the core building blocks of CSS layout — from the box model to positioning, navigation bars, and images.



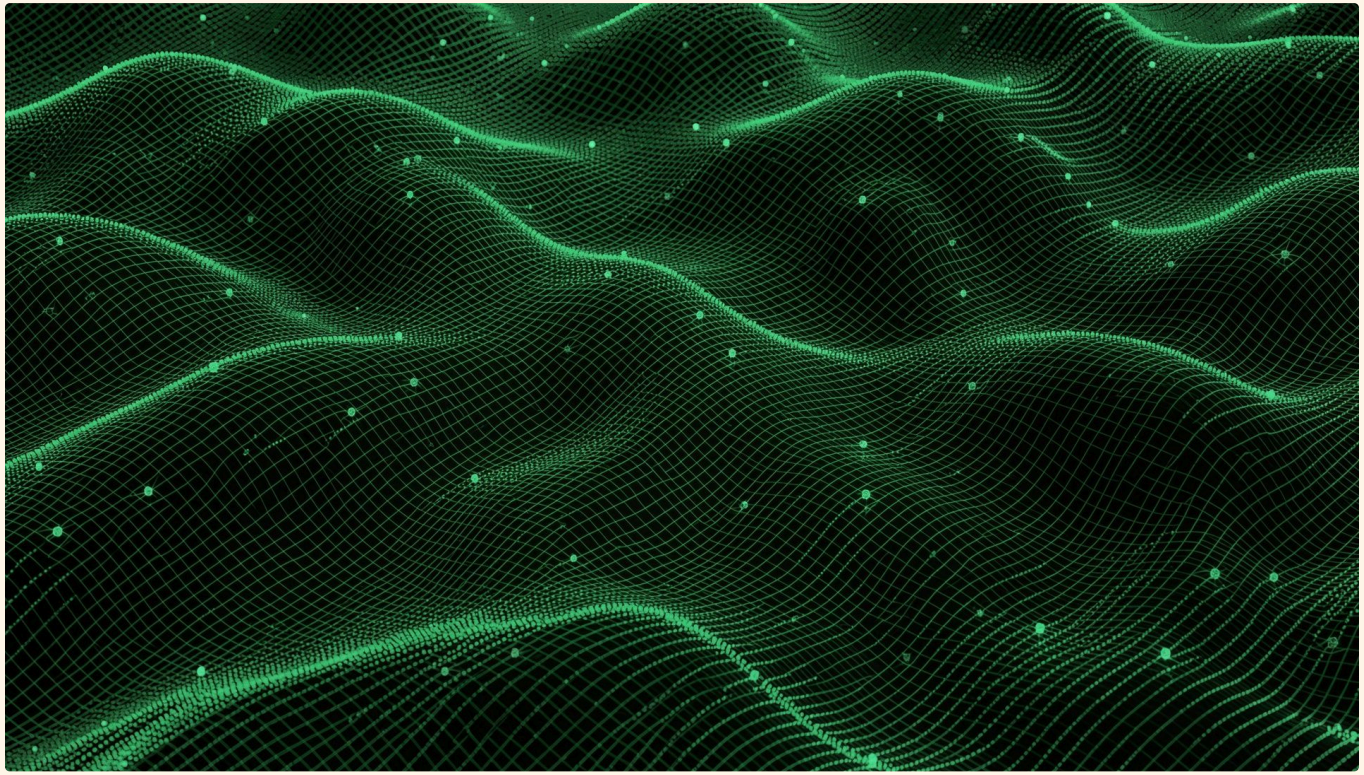


The Box Model: Every Element Is a Box

Every HTML element is a rectangular box made of four layers:

1. **Content**
   The actual text or image inside
2. **Padding**
   Space inside, between content and border
3. **Border**
   The visible edge wrapping the element
4. **Margin**
   Space outside — distance from neighbors

Margin is **outside** the box; padding is **inside** — this distinction drives every layout decision.



SPACING

Margin: Space Between Elements

Definition: Space *outside* an element — creates distance from surrounding elements. Think of it as the gap between boxes on a shelf.

Or use the shorthand: `margin: top right bottom left;`

Margin does *not* increase the element's content area.

margin-top	margin-right	margin-bottom	margin-left

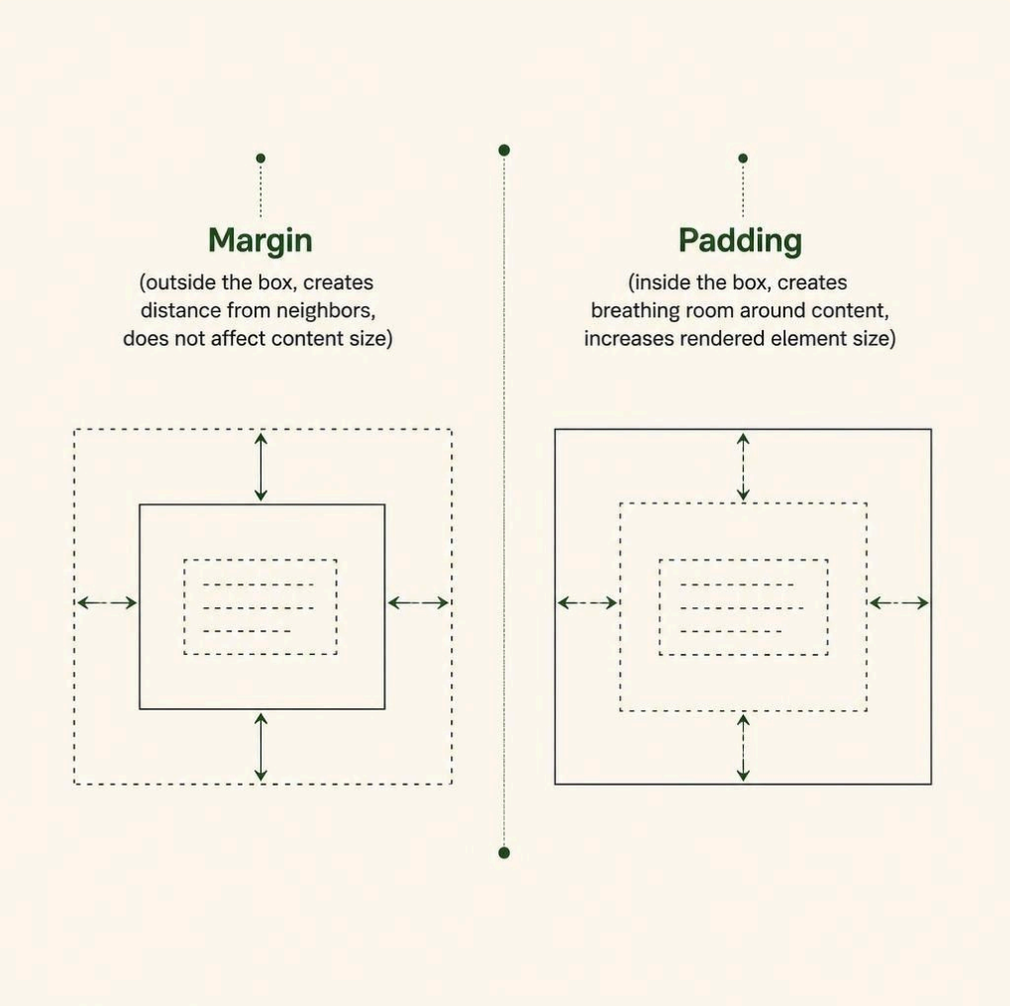
SPACING

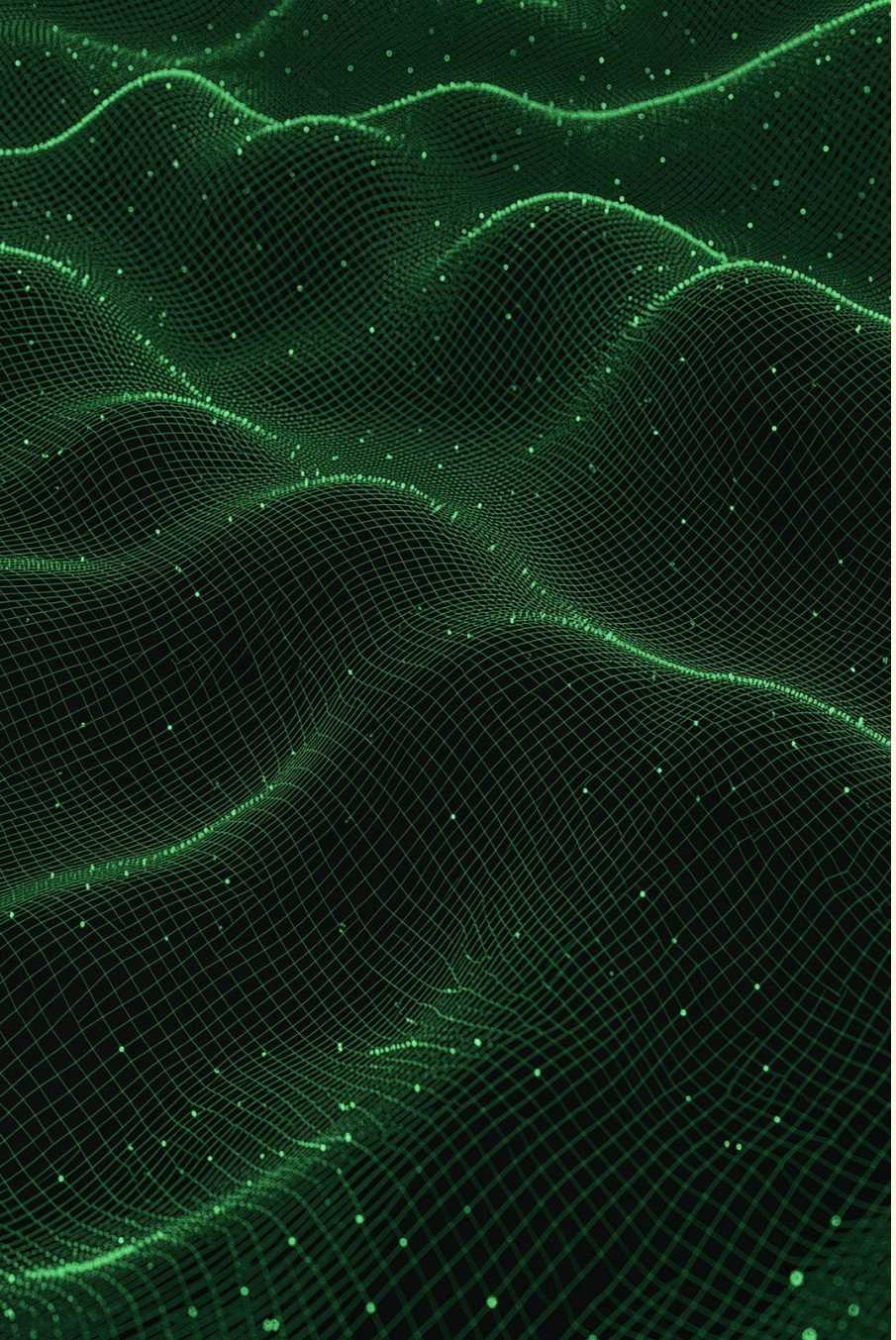
Padding: Space Inside an Element

Definition: Space between the content and the border — creates internal breathing room. Padding makes content more comfortable to read, like text inside a button.

Under the default box model, padding *increases* the element's rendered size.

> **Memory trick:** Margin = Outside · Padding = Inside





SPACING

Border: The Visible Edge

Three Core Properties

border-width · border-style · border-color

Shorthand: border: 2px solid black;

Common Styles

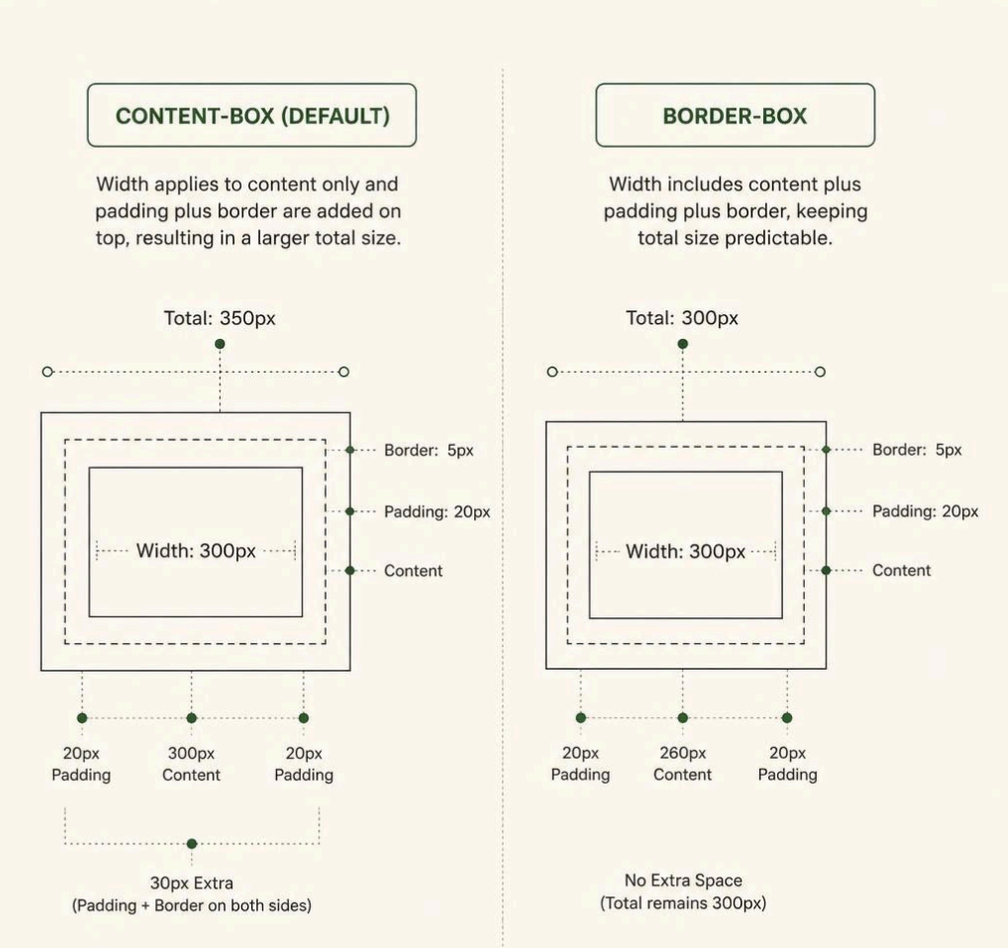
solid · dashed · dotted · double · groove · ridge · inset · outset · none

border-radius

Rounds corners. Set to 50% on a square element to create a perfect circle.

BOX MODEL

Box Sizing: Who Owns the Width?



content-box (default)

Width applies to content only — padding and border are added on top. Can cause unexpected overflow.

border-box

Width includes content + padding + border. No surprises — what you set is what you get.

Best practice: `* { box-sizing: border-box; }` makes all layouts predictable.

DISPLAY

Display: How Elements Occupy Space



block

Starts on a new line, takes full width. Examples: `div`, `p`, `h1`, `section`



inline

Flows with text, no width/height control. Examples: `span`, `a`, `strong`



inline-block

Sits beside other elements but accepts width/height. Great for buttons and nav items.



none

Removes element from layout entirely. Unlike `visibility: hidden`, which still reserves space.

POSITIONING

Positioning: Controlling Element Placement

static

Default — follows normal document flow. `top` / `left` / etc. have no effect.

relative

Stays in flow but can be nudged. Acts as a reference point for absolute children.

absolute

Removed from flow. Positioned relative to the nearest positioned ancestor.

fixed

Anchored to the viewport — stays put while the page scrolls. Ideal for nav bars and chat buttons.

sticky

Normal until scroll threshold, then sticks. Ideal for table headers and section labels.

POSITIONING PATTERN

The relative + absolute Pattern

The most common positioning pattern in CSS: the **parent** is set to `position: relative`, and the **child** is set to `position: absolute`. This allows precise placement of badges, icons, or overlays inside a container.

How It Works

The relative parent becomes the coordinate origin. The absolute child is then placed using `top`, `right`, `bottom`, or `left` — measured from the parent's edges, not the page.

```
.card {
  position: relative;
}

.badge {
  position: absolute;
  top: 10px;
  right: 10px;
  background: #38512F;
  color: white;
  padding: 4px 10px;
  border-radius: 12px;
}
```

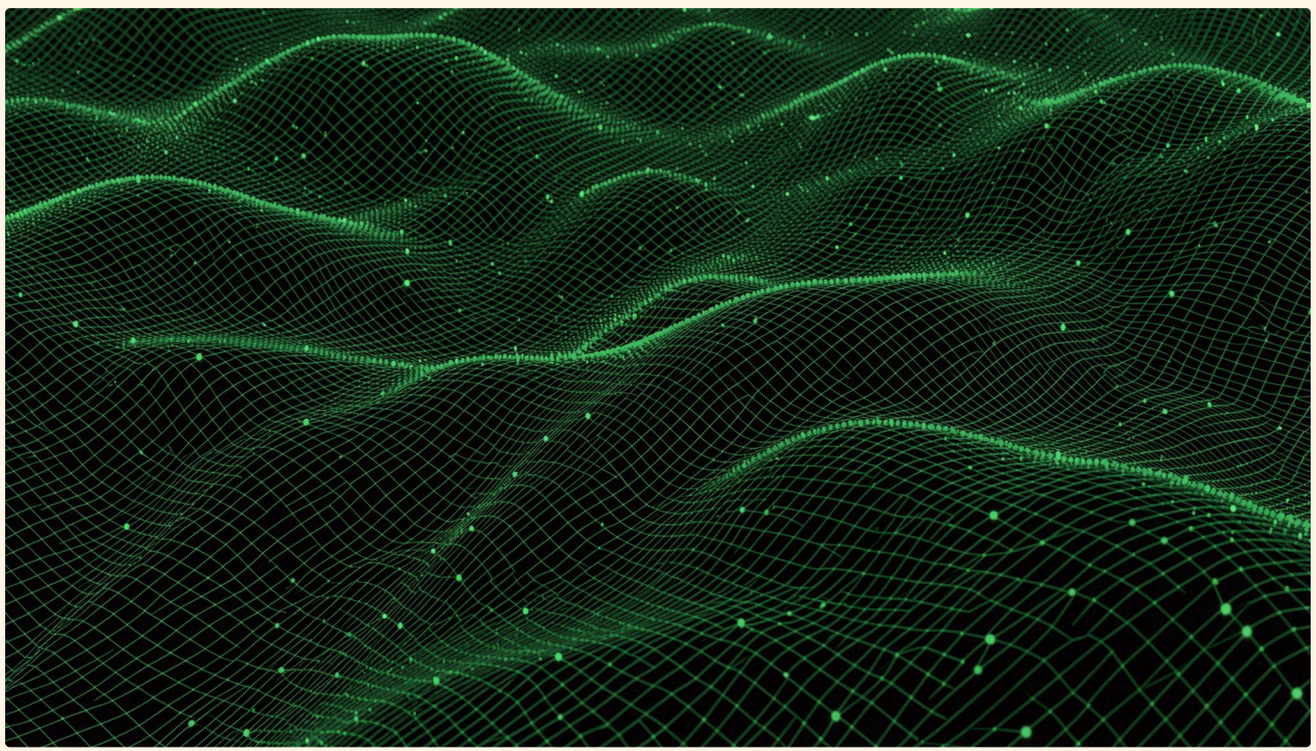
Visual Example: POPULAR Badge

A destination card uses `position: relative`. The "POPULAR" label is a child `span` with `position: absolute; top: 10px; right: 10px;` — it floats in the top-right corner of the card regardless of card content.

This exact pattern is used in the Lab task — you'll add a **POPULAR** badge to one of your Travel Explorer destination cards.

CSS IN ACTION

Navigation Bar: CSS Concepts in Action



A nav bar is not a special CSS element — it's a `<nav>` with links styled using CSS. Every concept you've learned applies here at once.

Layout

Links arranged horizontally with `display: inline-block` or a flex container

Spacing

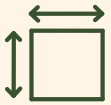
Padding creates click targets; margin separates links

Sticky

Use `position: sticky; top: 0;` to keep the nav visible while scrolling

IMAGES

Images in CSS



Sizing

Set width and height via CSS. Avoid setting both independently — let one be `auto` to prevent distortion.



Circular Avatar

`border-radius: 50%` on a square image creates a perfect circular avatar.



object-fit: cover

Fills the box completely — may crop edges. Best for hero images and cards.



object-fit: contain

Shows the full image — may leave gaps. Best when the full image must be visible.

SYNTHESIS

How It All Connects: The Profile Card

Margin

Separates cards from each other on the page

Border + Padding

Border defines the card boundary; padding creates comfortable space inside

Box Sizing

Controls how card dimensions are calculated — use `border-box`

Display

Controls how cards and their contents participate in layout flow

Positioning + object-fit

Places badges or icons over the card; styles the image inside it

INSTRUCTOR GUIDE

Recommended Teaching Order

01

Spacing First

Margin → Padding → Border — build intuition for space before structure

02

Box Model

Combine the three into one unified mental model

03

Box Sizing

content-box vs. border-box — why `border-box` wins

04

Display

block → inline → inline-block → none

05

Positioning

static → relative → absolute → fixed → sticky

06

Nav Bar

Combine display + spacing + positioning in one real component

07

Images

width + height + border-radius + object-fit

🧑‍💻 LAB TASK

Build a "Travel Explorer" Page

Header

Sticky nav bar — **Travel Explorer | Home | Destinations | Packages | Gallery | Contact**

Main Content

3 destination cards — each with image, name, description, price, and a button.

Requirements Checklist

- `margin`, `padding`, `border`, `border-radius`
- `box-sizing: border-box`
- `display` (block, inline-block)
- `position: relative` / `absolute` / `sticky`
- `object-fit` on card images

> **Challenge:** Add a **"POPULAR"** badge to one card using `position: absolute` inside a `position: relative` parent.

